Fixing PID

Proportional-integral-derivative controllers may be ubiquitous, but they're not perfect.

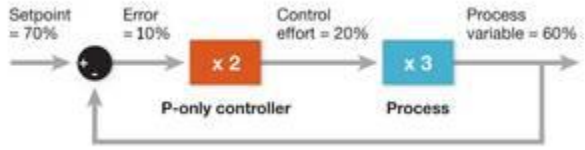
Vance VanDoren, PhD, PE for Control Engineering
11/30/2012

Although the PID algorithm was first applied to a feedback control problem more than a century ago and has served as the de facto standard for process control since the 1940s, PID loops have seen a number of improvements over the years. Mechanical PID mechanisms have been supplanted in turn by pneumatic, electronic, and computer-based devices; and the PID calculation itself has been tweaked to provide tighter control.

Integral action or automatic reset was the first fix introduced to improve the performance of feedback controllers back when they were equipped with only proportional action. A basic "P-only" controller applies a corrective effort to the process in proportion to the difference or error between the desired setpoint and the current process variable measurement.

When the setpoint goes up or the process variable goes down, the error between them grows in the positive direction, causing a P-only controller to respond with a positive control effort that drives the process variable back up towards the setpoint. In the converse case, the error grows in the negative direction and the controller responds with a negative control effort that drives the process variable back down.

A P-only controller is intuitive and easy to implement, but it tends to reach a point of diminishing returns. As the error decreases with each new control effort, so does the next control effort. That in turn slows the rate at which the error decreases until it ceases to change altogether. Unfortunately, that steady-state error is never zero. A P-only controller always leaves the process variable slightly off the setpoint. See the "Steady-state error" graphic for a demonstration of why this happens.



In this simple example of a generic control loop, a proportional-only controller with a gain of 2 drives a process with a steady-state gain of 3. That is, the controller amplifies the error by a factor of 2 to produce the control effort, and the process multiplies the control effort by a factor of 3 (and adds a few transient oscillations) to produce the process variable. If the setpoint is 70%, the process variable will end up at 60% after the transients die out. The error remains nonzero, yet the controller stops trying to reduce it any further.

Integral action operating in parallel with proportional action eliminates that steady-state error. It increases the control effort in proportion to the running total or integral of all the previous errors and continues to grow so long as any error remains. As a result, a proportional-plus-integral or "PI" controller never gives up. It continues to apply an ever-increasing control effort until the process variable reaches the setpoint.

Unfortunately, the introduction of integral action causes other problems that require their own fixes. Arguably the most significant is an increased risk of closed-loop instability where the integral action's persistence does more harm than good. It can grow so large that it forces the process variable to overshoot the setpoint.

If the controlled process happens to be particularly sensitive to the controller's efforts, that overshoot will cause an even larger error in the opposite direction. The controller will eventually change course in an effort to eliminate the new error but will end up driving the process variable back the other way, ending up even further from the setpoint. Eventually, the controller will start oscillating from fully on to fully off in a vain effort to reach a steady state.

The simplest solution to this problem is to reduce the amplification factor or gain by which the controller multiplies the integrated error to generate the integral action. On the other hand, reducing the integral gain risks increasing the time required for the closed-loop system to reach a steady state with zero error. Hundreds of analytical and heuristic techniques have been developed over the years to determine values for the integral and proportional gains that are just right for a particular application.

Reset windup

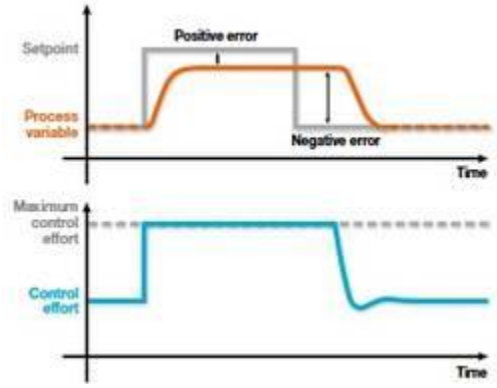
Integral action can also cause reset windup when the actuator is too small to implement an especially large control effort requested by the controller. That can happen when a burner isn't large enough to supply the necessary heat, a valve is too small to generate a high enough flow rate, or a pump reaches its maximum speed. The actuator is said to be saturated at some limiting value—either its maximum or minimum output.

When actuator saturation prevents the process variable from rising any further, the controller continues to see positive errors between the setpoint and the process variable. The integrated error continues to rise, and the integral action continues to call for an increasingly aggressive control effort. However, the actuator remains stuck at its maximum output, unable to affect the process any further, so the process variable doesn't get any closer to the setpoint.

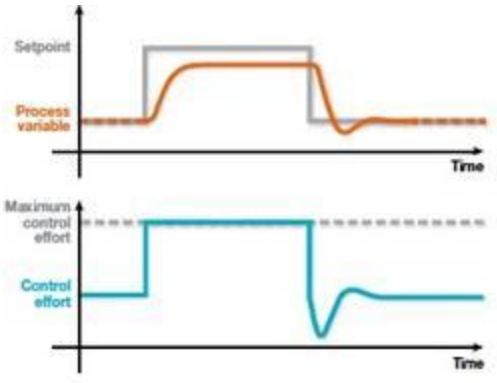
An operator can try to fix the problem by reducing the setpoint back into the range that the actuator is capable of achieving, but the controller will not respond because of the enormous value that the total integrated error will have achieved during the time that the actuator was saturated fully on. That total will remain very large for a very long time, no matter what the current value of the error happens to be. As a result, the integral action will remain very high, and the controller will keep pushing the actuator against its upper limit.

Fortunately, the error will turn negative if the operator drops the setpoint low enough, so the total integrated error will eventually start to fall. Still, a long series of negative errors will be required to cancel the long series of positive errors that had been accumulating in the integrator's running total. Until that happens, the integral action will remain large enough to continue saturating the actuator, no matter what the simultaneous proportional action is calling for. See the "Reset windup" graphic.

Several techniques have been devised to protect against reset windup. The obvious solution is to replace the undersized actuator with a larger one capable of actually driving the process variable all the way to the setpoint or selecting setpoints that the existing actuator can actually reach. Otherwise, the integrator can simply be turned off when the actuator saturates.



In this example, the operator has tried to increase the setpoint to a value higher than the actuator is capable of achieving. After observing that the controller has been unable to drive the process variable that high, the operator returns the setpoint to a lower value. Note that the controller does not begin to lower its control effort until well after the setpoint has been lowered because the integral action has grown very large during the controller's futile attempt to reach the higher setpoint. Instead, the controller continues to call for a maximum control effort even though the error has turned negative. The control effort does not begin to drop until the negative errors following the setpoint change have persisted as long as the positive errors had persisted prior to the setpoint change (or more precisely, until the integral of the negative errors reaches the same magnitude as the integral of the positive errors).



In this case, the operator has repeated the same sequence of setpoint moves, but this time using a PID controller equipped with reset windup protection. Extra logic added to the PID algorithm shuts off the controller's integrator when the actuator hits its upper limit. The process variable still can't reach the higher setpoint, but the controller's

integral action doesn't wind up during the attempt. That allows the controller to respond immediately when the setpoint is lowered.

Pre-loading

Reset windup also occurs when the actuator is turned off but the controller isn't. In a cascade controller, for example, the outer-loop controller will see no response to its control efforts while the inner loop is in manual mode. If the outer-loop controller is left operating during that interval, its integral action will "wind up" as the error remains constant.

Similarly, when the burner, valve, or pump is shut down between cycles of a batching operation, the process variable is prevented from getting any closer to the setpoint, again leading to windup. That's not a problem while the actuator remains off, but as soon the actuator is reactivated at the beginning of the next batch, the controller will immediately call for a 100% control effort and saturate the actuator.

The obvious solution to this problem is to turn off the controller's integrator whenever the actuator is turned off or to eliminate the error artificially by adjusting the setpoint to whatever value the process variable takes between batches. But there's another approach that not only prevents reset windup between batches but actually improves the controller's performance during the next batch.

Pre-loading freezes the output of the controller's integrator between batches so that the integral action starts the next batch with the total integrated error that it had accumulated as of the end of the previous batch. This technique assumes that the controller is eventually going to need the same amount of integral action to reach the same steady state as in the previous batch, so there's no point in starting the integrated error at zero. With pre-loading, the integral action essentially picks up right where it left off at the end of the previous batch, thereby shortening the time required to achieve a steady state in the next batch.

Pre-loading works best if each successive batch is more-or-less identical to its predecessor so that the controller is attempting to achieve the same setpoint under the same load conditions every time. But even if the batches aren't identical, it is sometimes possible to use a mathematical model of the process to predict what level of integral action is eventually going to be required to achieve a steady state. The required integrated error can then be deduced and pre-loaded into the controller's integrator at the start of each batch. This approach will also work for a continuous process if it can be modeled prior to the initial start-up.

Bumpless transfer

One potential drawback to pre-loading is the abrupt change it can cause in the actuator's output at the beginning of each batch. Although the actuator won't immediately slam all the way open if pre-loading is used to prevent reset windup, it will try to start the next batch at or near the position it was in at the end of the previous batch. If that causes an abrupt change that is likely to damage the actuator, the controller's integrator can be ramped up gradually to its pre-load value.

A similar problem can occur when a controller is switched from automatic to manual mode and back again. If an operator manually modifies the control effort during that interval, the controller will abruptly switch the control effort to whatever the PID algorithm is calling for at the time automatic mode is resumed.

Bumpless transfer solves this problem by artificially pre-loading the integrator with whatever value is required to restart automatic operations without changing the control effort from wherever the operator left it. The controller will still have to make adjustments if the operator's actions, changes in the process variable, or changes in the setpoint have created an error while the controller was in manual mode, but less of a "bump" will result when the controller is "transferred" back to automatic mode.

Still more windup problems

All of these windup-related problems are compounded by deadtime—the interval required for the process variable to change following a change in the control effort. Deadtime typically occurs when the process variable sensor is located too far downstream from the actuator. No matter how hard the controller works, it cannot begin to reduce an error between the process variable and the setpoint until the material that the actuator has modified reaches the sensor.

As the controller is waiting for its efforts to take effect, the process variable and the error will remain fixed, causing the integral action to wind up just as if the actuator were saturated or turned off. The most common fix to this problem is to de-tune the controller; that is, reduce the integral gain to reduce the maximum integral action caused by windup.

The same effect can be achieved by equipping the integral action with its own deadtime so that windup does not begin until at least some of the process deadtime has elapsed. This in turn lowers the total integrated error that the controller can accumulate during the interval that the error is fixed.

Deadtime-induced windup can also be ameliorated by making the integral action intermittent. Let the proportional action do all the work until the process variable has settled somewhere close to the setpoint, then turn on the integral action only long enough to eliminate the remaining steady-state error. This approach not only delays the onset of windup, it gives the integral action only small errors to deal with, thereby reducing the maximum windup effect.

But wait, there's more

This only scratches the surface of the many ways engineers have sought to improve control performance. The PID algorithm has also been modified to deal with velocity-limited actuators, time-varying process models, noise in the process variable measurement, excessive derivative action during setpoint changes, and more. Future installments of this series will look at the effects these problems cause and how they can be avoided.

Vance VanDoren, PhD, PE, is Control Engineering *contributing content specialist. Re*

Fixing PID, Part 2

Proportional-integral-derivative controllers may be ubiquitous, but they’re not perfect.

Vance VanDoren, PhD, PE
04/28/2014

(*Control Engineering*, Nov. 2012) looked at several issues that limit the performance of the theoretical PID algorithm in real-world applications of feedback control.

All of these problems are exacerbated by uncertainty. Sometimes the controller lacks sufficient information about the controlled process to know how much and how long to apply a control effort. Sometimes the controller can't even tell if it's done a good job or how to do a better job in the future when sensor placement, physical limitations of the sensing technology, or measurement noise make the process variable hard to measure.

Measurement noise is particularly troublesome for a PID controller's derivative action. To compute the "D" component of its next control effort, the controller computes the latest change in the error (the difference between the process variable and the setpoint) and multiplies that by the derivative gain or rate parameter.

When random electrical interference or other glitches in the sensor's output cause the controller to see fictitious changes in the process variable, the controller's derivative action increases or decreases unnecessarily. If the noise is particularly severe or the derivative gain is particularly high, the controller's subsequent chaotic control efforts may be not only unnecessary, but also damaging to the actuator and perhaps even to the controlled process itself.

Filtering

The simplest solution to this problem is to reduce the derivative gain when measurement noise is high, but doing so limits its effectiveness. The measurement noise itself can sometimes be reduced by fixing the sensor or by filtering the process variable measurement mathematically. A process variable filter essentially averages the sensor's most recent outputs to produce a better estimate of the process variable's actual value.

However, process variable filters have their limitations. They only work if the measurement noise is truly random, sometimes increasing and sometimes decreasing the sensor's output in equal measure. If those positive and negative blips also occur with equal frequency, then the filter's averaging operation will tend to cancel them out. But if the measurement noise tends to skew the sensor's output consistently in one direction or the other, the filtered process variable will tend to run consistently too high or too low, thereby deceiving the controller into working too hard or too little.

A process variable filter also slows the controller's reaction time. If the filter is configured to average a particularly long sequence of sensor outputs, it will do a better job of cancelling out random blips, but it will also tend to miss the most recent changes in the actual process variable. The filter needs to see a sustained change in the sensor's output before it can report a new value of the process variable to the controller. The controller can't even see, let alone react to rapid, short-term changes in the process variable, as discussed in the "Filtering" section below.

As a compromise, some PID controllers can be configured to filter the process variable to differing degrees when computing the proportional, integral, and derivative actions. The derivative action requires the most filtering since that's where measurement noise causes the most problems. The proportional action may benefit from less filtering (that is, a filter incorporating a shorter sequence of sensor outputs) in order to remain responsive to short-term changes in the process variable. And since the integral action itself serves as a filter, it may require no process variable filtering at all.

Alternately, a filter can be applied to the control effort instead of the process variable. Doing so permits the measurement noise to enter into the PID calculations (especially the "D"), but the noisy control effort that results is smoothed by the filter before reaching the actuator. A filter can also help slow down the control effort to prevent overly dramatic fluctuations in the process's behavior in cases where the process is particularly sensitive to the actuator's movements.

On the other hand, a filter on the control effort can make the process appear more sluggish than it really is. An operator looking for faster closed-loop performance might try re-tuning the controller to be more aggressive without realizing that the problem is the damping effect of the filter, not the process. The controller tuning and the control effort filter sometimes end up battling each other unnecessarily when different operators have implemented one without checking for the other.

Deadband

The effects of measurement noise can also be mitigated by simply ignoring insignificant changes in the sensor's output under the assumption that they're probably just artifacts of the measurement noise and are too small to make a difference in the controller's choice of control efforts anyway. So long as the error between the process variable and the setpoint remains within a range known as the deadband, the controller simply does nothing.

The trick is determining how much of a change in the error is small enough to ignore. If the deadband is set too large, significant changes in the behavior of the process may be overlooked. But if it is set too small, the controller will react unnecessarily to every fictitious blip in the sensor's output, even if the actual process variable has already reached the setpoint.

Unfortunately, a deadband also glosses over small changes in the setpoint. If an operator tries to move the process into a higher or lower operating range that falls within the current deadband, the resulting change in the error will be ignored and the controller will do nothing. If the deadband is too large, the controller's precision will suffer. That is, it may be able to make a refrigerated space five degrees colder, but not just one.

Fixing PID, Part 3

PID: Proportional-integral-derivative controllers may be ubiquitous, but they’re not perfect. Consider how deadtime affects integral action.

Vance VanDoren, PhD, PE

11/25/2015

In proportional-integral-derivative (PID) control methods, the actions of the integral, the "I" term in PID, can be particularly problematic because the controller's integrator continues to build up an ever-increasing control effort so long as an error remains between the measured value of the process variable and the desired setpoint. Look at the related challenges and advice that follow for more practical help to fix PID.

Previous installments of this series—"Fixing PID, part 1" (*Control Engineering*, November 2012) and "Fixing PID, part 2" (*Control Engineering*, May 2014) linked below, online—have examined the limitations of PID controllers and the various patches engineers have developed to fix them.

With a PID controller, a particularly aggressive integral action can cause the controller to oscillate back and forth, hunting in vain for a control effort that will force the process variable to match the setpoint. The integral action also has trouble dealing with a process that does not respond immediately to the controller's efforts. The longer the delay, or deadtime, the greater the integrated error becomes before the process variable starts to move. That in turn causes the integral action to work harder and harder, potentially to the point of doing more harm than good.

Rate-limited actuators

Fortunately, these tend to be especially obvious problems that an experienced control engineer can spot and fix readily, either by scaling back the integral action or by modifying the process to be easier to control. A more insidious issue arises when the actuator responsible for applying the controller's efforts to the process is rate-limited.

A rate-limited actuator is one that can't act as fast as the controller wants it to, usually as a result of mechanical limitations. A valve, for example, can only open or close as fast as the actuator can move the stem. It tends to fall behind when the controller gives it a command to open or close too rapidly.

The actuator's rate limit will in turn limit how fast the process variable can change, leaving a sustained error between the process variable and the setpoint. Just as when deadtime prevents the process variable from changing at all, the integrated error will continue to grow as will the integral action. That makes matters worse and worse as the actuator falls further and further

behind. This reset windup effect is less pronounced when caused by rate limiting rather than deadtime, but unstable oscillations in the process variable result either way, as explained in the "Effects of deadtime and a rate-limited actuator" diagram and sidebar at the bottom of this article.

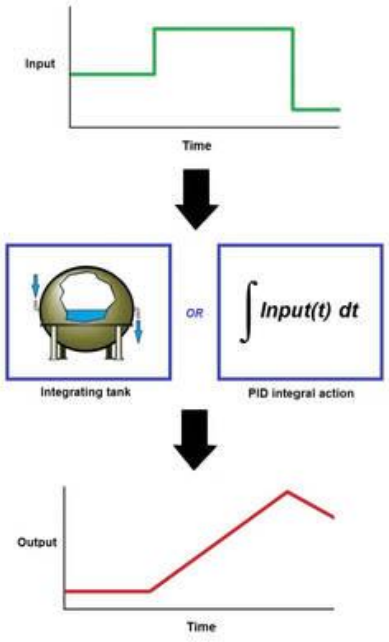
The insidious part is that the effects of a rate-limited actuator don't always show up immediately. If the controller's commands are fast but small, the actuator might still be able to keep up for a while. The controller might even be able to maintain the process variable at the setpoint for years, so long as any disturbances that move the process variable off its mark happen to remain modest. But if a large enough disturbance comes along requiring a commensurately large control effort, the integral action will start to wind up, and hunting will ensue unexpectedly.

External reset to the rescue

Fortunately, there are three simple fixes for a rate-limited actuator. The most obvious is to replace the sluggish actuator with a faster one capable of implementing the controller's commands in a timely manner. Continuing the example, sometimes cleaning the stem of a gunked-up valve to reduce rate-limiting friction is enough.

Conversely, the integral action can be slowed to the point that the controller no longer asks for impossibly fast actuator moves when reacting to large disturbances. However, doing so also retards the controller's reaction to small disturbances that aren't really a problem.

Arguably the most effective solution is external reset. An external reset function modifies the rate at which the error is integrated so that the speed of the integral action matches the speed of the actuator. The result is a controller that seems to know just how fast it can afford to push the actuator. External reset can also combat other forms of reset windup no matter how they are caused. Part 1 of this series discusses that phenomenon in greater detail.



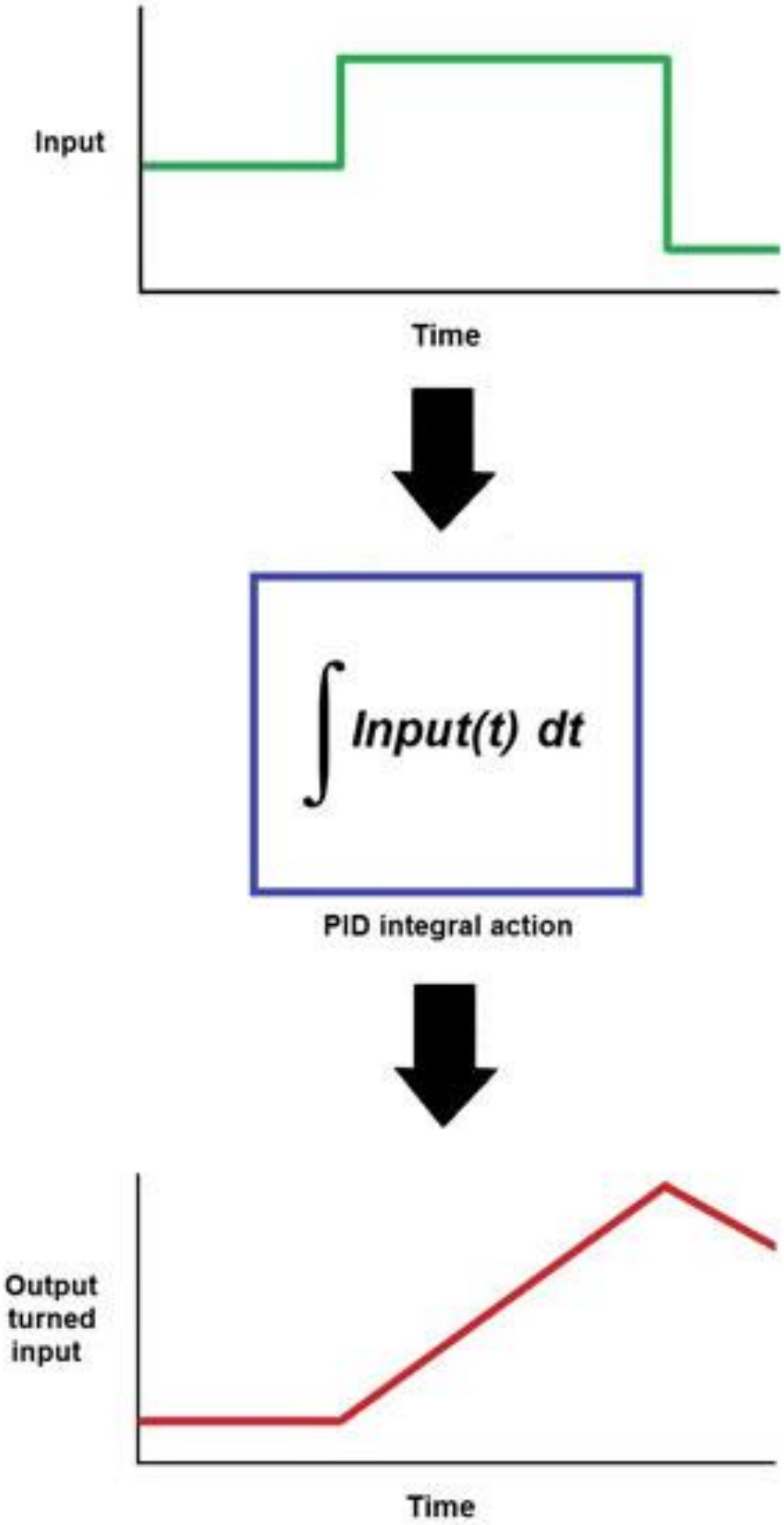
Integrating processes

A PID controller's integral action can also cause unstable oscillations when the controlled process itself acts as an integrator. Consider, for example, the level of water in a tank equipped with an inflow and an outflow as described in "Fundamentals of integrating vs. self-regulating processes" (*Control Engineering*, December 2014). Every drop that enters without being removed accumulates in the tank, causing the level to rise. The tank integrates its input (the net inflow minus outflow) to generate its output (the tank level).

The integral action embedded in a PID controller employs a purely mathematical integrator to perform the same conceptual function. It sums all past values of the error between the process variable and the setpoint to generate a numerical output that increases as long as the process variable is less than the setpoint and decreases as long as the process variable is greater than the setpoint. This *integrated error* is multiplied by a gain to generate the integral action's control effort.

Both the tank and the controller's integrator react to step changes in their inputs by ramping up their outputs as shown in Figure 2, "Back-to-back integrators." Connecting the two in series as shown in Figure 3 leads to an exponential increase in the tank's output when the controller's input is stepped up. This behavior is typical of back-to-back integrators, which is why integral control applied to an integrating process often leads to unstable results.

Loop tuning tradeoffs


Input
Time
$\int Input(t)\ dt$
PID integral action
Output turned input
Time

As in the case of rate-limited actuators and back-to-back integrators, many of the patches that have been proposed to fix PID problems involve configuring the controller to be less aggressive in reacting to errors between the process variable and the setpoint. Unfortunately, that configuration operation, known as loop tuning, can itself be a problem.

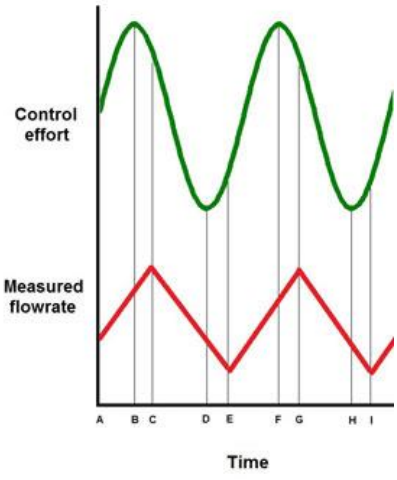
In fact, loop tuning is arguably the most difficult aspect of designing a PID controller because the appropriate choices for the P, I, and D parameters required to achieve just the right mix of the three actions are rarely obvious. Even experienced control engineers can find it challenging to observe the controller in action and decide which of those parameters needs to be increased or decreased, let alone by how much.

For example, the derivative action can be used as a brake to prevent rapid changes in the control effort as the controller is trying to bring the process variable in for a landing right on the setpoint. Without it, a controller with especially aggressive integral and proportional action can drive the process variable towards the setpoint so rapidly that it overshoots. That in turn will cause the controller to overcorrect in the opposite direction, potentially overshooting back and forth again and again.

On the other hand, the derivative action also gives the process an immediate kick at the outset of a disturbance response. That kick can help drive the process variable in the right direction without waiting for the integral and proportional actions to ramp up their efforts. However, if the derivative action is tuned to be especially aggressive for the sake of braking at the end of a disturbance response, it can also be too aggressive in kicking the process at the beginning. This can cause overshoot too, particularly when the controlled process is particularly sensitive to the controller's efforts.

Loop tuning software helps

Fortunately, innumerable formulas and software packages have been developed to facilitate loop tuning. Control engineers do not need to rely solely on qualitative observations of the controller in action. They can accomplish a great deal with quantitative measurements of the process's open and closed-loop behavior. See "Loop tuning fundamentals" (*Control Engineering*, July 2003).



Effects of deadtime and a rate-limited actuator

A PID controller can cause hunting when its actuator is subject to deadtime and/or rate limitations. In this flow control example, an especially aggressive controller is using a rate-limited valve to maintain the flowrate downstream. At time **A**, it commands the valve to open rapidly in an attempt to combat a drop in the flowrate cause by a load disturbance. The valve responds as fast as it can between times **A** and **B**, but not as fast as the controller expects. Thinking that its efforts have been inadequate, the controller starts to push the valve even harder, but to no avail. At time **B**, it reverses course after detecting that it has overcorrected for the original disturbance. The actual flowrate begins to drop as soon as the controller commands the valve to start closing, but since the flowmeter is located some distance downstream, the controller continues to see the flowrate increasing until time **C**. By then, the controller has already begun to undershoot the setpoint in the opposite direction. This sequence repeats at times **D** and **E, F, G, H,** and **I**, etc. Courtesy: *Control Engineering*

- Vance VanDoren, PhD, PE, is a Control Engineering contributing content specialist. Reach him at controleng@msn.com. Edited by Peter Welander, *Control Engineering*, pwelander@cfemedia.com.

Key concepts:

- PID control can be found everywhere, but few understand its peculiarities.
- Understanding how it works can help unravel problems and improve tuning.

Consider this

How many of your control loops are open? How much more efficient could you be with closed-loop control?